Assignment - 1

Code-

Recursive

```
#include <iostream>
using namespace std;

int fibonacci(int n) {
    if (n <= 1)
        return n;

    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n;

    cout << "Enter the number of terms: ";
    cin >> n;

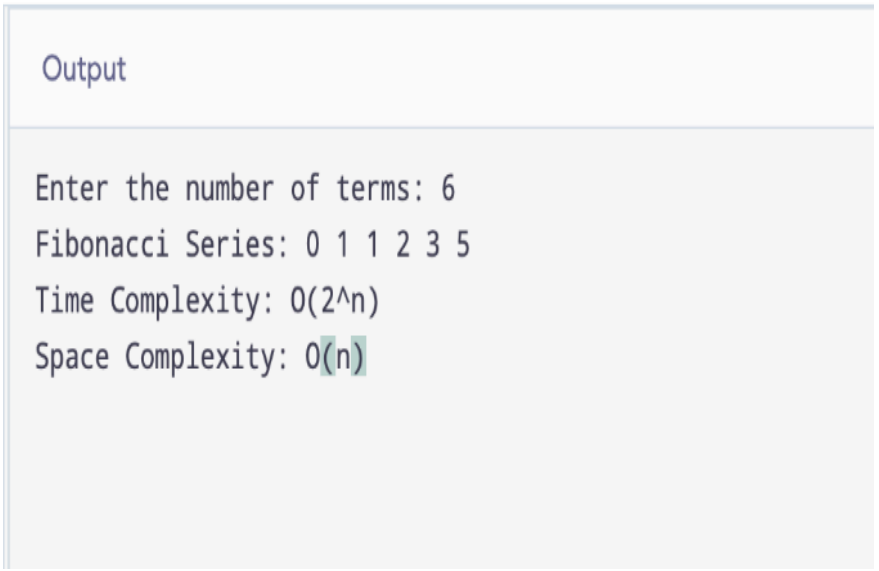
    if (n <= 0) {
        cout << "Enter a positive number.";
        return 0;
    }

    cout << "Fibonacci Series: ";
    for (int i = 0; i < n; i++)
        cout << fibonacci(i) << " ";

    cout << "\nTime Complexity: O(2^n)";
    cout << "\nSpace Complexity: O(n)";

    return 0;
}
```

Output -



Code -

Non - Recursive

```
#include <iostream>
using namespace std;

int fibonacci(int n) {
    if (n <= 1)
        return n;

    int a = 0, b = 1, c;

    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }

    return b;
}
```

```
int main() {
    int n;

    cout << "Enter the number of terms: ";
    cin >> n;

    if (n <= 0) {
        cout << "Enter a positive number.";
        return 0;
    }

    cout << "Fibonacci Series: ";
    for (int i = 0; i < n; i++)
        cout << fibonacci(i) << " ";

    cout << "\nTime Complexity: O(n)";
    cout << "\nSpace Complexity: O(1)";

    return 0;
}
```

Output -

Output

```
Enter the number of terms: 8
Fibonacci Series: 0 1 1 2 3 5 8 13
Time Complexity: O(n)
Space Complexity: O(1)
```

Assignment - 2

Code -

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

struct Node {
    char ch;
    int freq;
    Node *left, *right;

    Node(char c, int f) {
        ch = c;
        freq = f;
        left = right = nullptr;
    }
};

struct Compare {
    bool operator()(Node* a, Node* b) {
        return a->freq > b->freq;
    }
};

void printCodes(Node* root, string code) {
    if (root == nullptr)
        return;

    if (root->left == nullptr && root->right == nullptr) {
        cout << root->ch << " : " << code << endl;
        return;
    }
```

```
    printCodes(root->left, code + "0");
    printCodes(root->right, code + "1");
}

int main() {
    int n;
    cout << "Enter number of characters: ";
    cin >> n;

    if (n <= 0) {
        cout << "Invalid number of characters.";
        return 0;
    }

    priority_queue<Node*, vector<Node*>, Compare> pq;

    cout << "Enter character and frequency:\n";

    for (int i = 0; i < n; i++) {
        char ch;
        int freq;
        cin >> ch >> freq;

        if (freq <= 0) {
            cout << "Frequency must be positive.";
            return 0;
        }

        pq.push(new Node(ch, freq));
    }

    while (pq.size() > 1) {
        Node* left = pq.top();
        pq.pop();

        Node* right = pq.top();
```

```
        pq.pop();

        Node* parent = new Node('$', left->freq + right->freq);
        parent->left = left;
        parent->right = right;

        pq.push(parent);
    }

    cout << "\nHuffman Codes:\n";
    printCodes(pq.top(), "");

    cout << "\nTime Complexity: O(n log n)";
    cout << "\nSpace Complexity: O(n)";

    return 0;
}
```

Output -

Output

```
Enter number of characters: 4
Enter character and frequency:
A 5
B 9
C 12
D 13

Huffman Codes:
A : 00
B : 01
C : 10
D : 11

Time Complexity: O(n log n)
Space Complexity: O(n)
```

Assignment - 3

Code -

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Item {
    int profit, weight;
    double ratio;
};

bool compare(Item a, Item b) {
    return a.ratio > b.ratio;
}

int main() {
    int n, capacity;
    cout << "Enter number of items: ";
    cin >> n;

    Item items[n];

    cout << "Enter profit and weight of each item:\n";
    for (int i = 0; i < n; i++) {
        cin >> items[i].profit >> items[i].weight;
        items[i].ratio = (double)items[i].profit / items[i].weight;
    }

    cout << "Enter knapsack capacity: ";
    cin >> capacity;

    sort(items, items + n, compare);

    double totalProfit = 0;
```

```
    for (int i = 0; i < n; i++) {
        if (capacity == 0)
            break;

        if (items[i].weight <= capacity) {
            totalProfit += items[i].profit;
            capacity -= items[i].weight;
        } else {
            totalProfit += items[i].ratio * capacity;
            capacity = 0;
        }
    }

    cout << "Maximum Profit: " << totalProfit;
    cout << "\nTime Complexity: O(n log n)";
    cout << "\nSpace Complexity: O(n)";

    return 0;
}
```

Output -

Output

```
Enter number of items: 3
Enter profit and weight of each item:
60 10
100 20
120 30
Enter knapsack capacity: 50
Maximum Profit: 240
Time Complexity: O(n log n)
Space Complexity: O(n)
```

Assignment - 4

Code -

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    int n, capacity;

    cout << "Enter number of items: ";
    cin >> n;

    int profit[n], weight[n];

    cout << "Enter profit and weight of each item:\n";
    for (int i = 0; i < n; i++) {
        cin >> profit[i] >> weight[i];
    }

    cout << "Enter knapsack capacity: ";
    cin >> capacity;

    int dp[n + 1][capacity + 1];

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (weight[i - 1] <= w)
                dp[i][w] = max(
                    profit[i - 1] + dp[i - 1][w - weight[i - 1]],
                    dp[i - 1][w]
                );
            else
                dp[i][w] = dp[i - 1][w];
```

```
        }
    }

    cout << "Maximum Profit: " << dp[n][capacity];
    cout << "\nTime Complexity: O(n * W)";
    cout << "\nSpace Complexity: O(n * W)";

    return 0;
}
```

Output -

Output

```
Enter number of items: 3
Enter profit and weight of each item:
60 10
100 20
120 30
Enter knapsack capacity: 50
Maximum Profit: 220
Time Complexity: O(n * W)
Space Complexity: O(n * W)
```

Assignment - 5

Code -

```
#include <iostream>
#include <vector>
using namespace std;

int N;

bool isSafe(vector<vector<int>>& board, int row, int col) {
    for (int i = 0; i < row; i++)
        if (board[i][col] == 1)
            return false;

    for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--)
        if (board[i][j] == 1)
            return false;

    for (int i = row - 1, j = col + 1; i >= 0 && j < N; i--, j++)
        if (board[i][j] == 1)
            return false;

    return true;
}

bool solveNQueens(vector<vector<int>>& board, int row) {
    if (row == N)
        return true;

    for (int col = 0; col < N; col++) {
        if (isSafe(board, row, col)) {
            board[row][col] = 1;

            if (solveNQueens(board, row + 1))
                return true;
```

```
            board[row][col] = 0;
        }
    }

    return false;
}

int main() {
    cout << "Enter the value of N: ";
    cin >> N;

    if (N <= 0) {
        cout << "Invalid value of N.";
        return 0;
    }

    vector<vector<int>> board(N, vector<int>(N, 0));
    bool found = false;

    for (int col = 0; col < N; col++) {
        board[0][col] = 1;

        if (solveNQueens(board, 1)) {
            found = true;
            break;
        }

        board[0][col] = 0;
    }

    if (found) {
        cout << "Solution:\n";

        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++)
                cout << board[i][j] << " ";
            cout << endl;
```

```
        }
    } else {
        cout << "Solution does not exist.";
    }

    cout << "\nTime Complexity: O(N!)";
    cout << "\nSpace Complexity: O(N^2)";

    return 0;
}
```

Output -

